



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

Operation System Assignment

Subject: Operating System

5th Semester

Faculty Name: Dr. Saneh Lata Yadav

Submitted By: Utsav Ratan

Roll No: 2401010046

Section: B. Tech CSE Core (B)

Experiment 1 – getpid() and getppid()

Description: -

This experiment demonstrates how a process obtains its own Process ID (PID) and the Process ID of its parent. The program uses `os.getpid()` for the current process and `os.getppid()` for the parent process.

```
xylanxd@LAPTOP-U3VEKHRD:~$ python3 --version
Python 3.14.4
xylanxd@LAPTOP-U3VEKHRD:~$ mkdir ~/os_system_calls
xylanxd@LAPTOP-U3VEKHRD:~$ cd ~/os_system_calls
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ pwd
/home/xylanxd/os_system_calls
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano process_id.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 process_id.py
```

Screenshot 1

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano process_id.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 process_id.py
Process ID: 789
Parent Process Id: 363
```

Screenshot 2

```
GNU nano 8.7.1 process_id.py
import os
print("Process ID:", os.getpid())
print("Parent Process Id:", os.getppid())

[ Read 3 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo     M-G Copy
```

Screenshot 3

Experiment 2 – Creating a Process Using fork()

Description: -

This experiment demonstrates creation of a child process using the Linux fork() system call. A return value of 0 identifies the child process, while a positive value in the parent represents the child's PID.

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano fork_demo.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 fork_demo.py
Before fork()
This is the parent process
Parent PID: 937
Child PID: 938
This is the child process
Child PID: 938
Parent PID: 937
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ |
```

Screenshot 1



```
GNU nano 8.7.1 fork_demo.py
import os
print("Before fork()")
pid = os.fork()
if pid == 0:
    print("This is the child process")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
else:
    print("This is the parent process")
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)
```

[Read 11 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

Screenshot 2

Experiment 3 – Parent Waiting for Child Using wait()

Description: -

This experiment demonstrates process synchronization. The parent uses `os.wait()` so that it waits for the child process to finish before continuing.

```
xylandx@LAPTOP-U3VEKHRD:~/os_system_calls$ nano wait_demo.py
xylandx@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 wait_demo.py
Parent is waiting for child...
Child process started
Child PID: 967
Child process has finished
```

Screenshot 1



```
GNU nano 8.7.1 wait_demo.py
import os
pid = os.fork()
if pid == 0:
    print("Child process started")
    print("Child PID:", os.getpid())
else:
    print("Parent is waiting for child...")
    os.wait()
    print("Child process has finished")
```

[Read 9 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

Screenshot 2

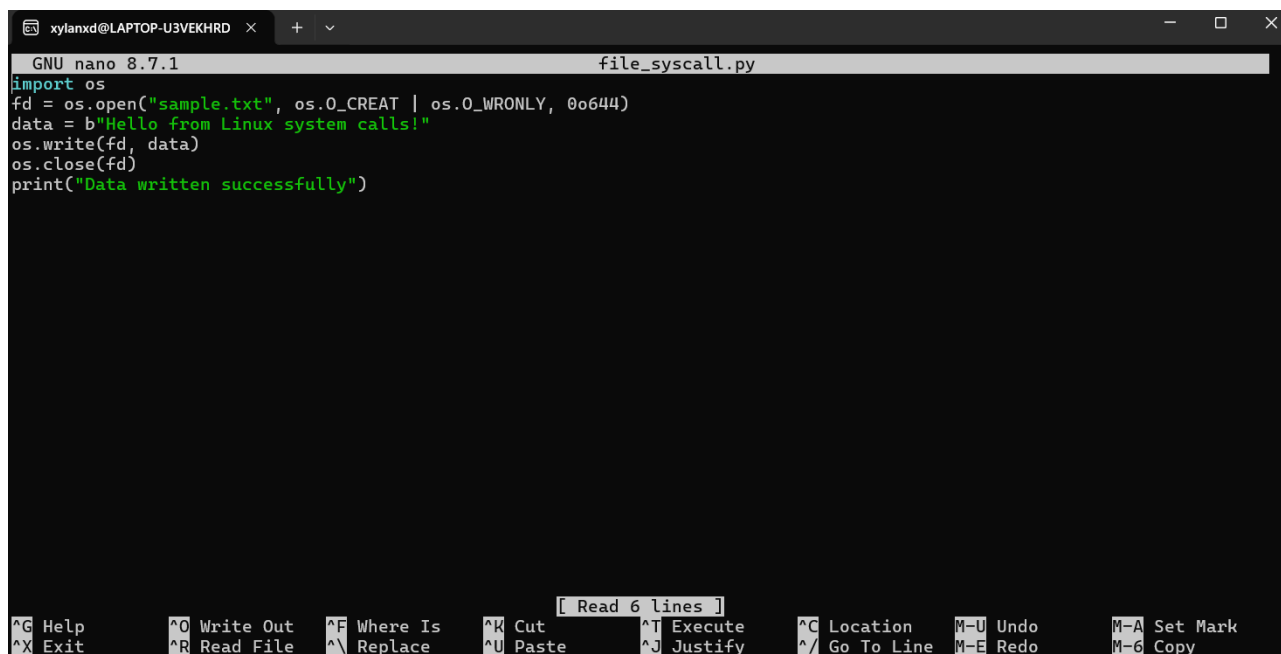
Experiment 4 – File System Calls

Description: -

This experiment demonstrates basic file system calls using `os.open()`, `os.write()`, and `os.close()`. The program creates `sample.txt` and writes a message to it.

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano file_syscall.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 file_syscall.py
Data written successfully
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ ls
file_syscall.py  fork_demo.py  process_id.py  sample.txt  wait_demo.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ cat sample.txt
Hello from Linux system calls!xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ |
```

Screenshot 1



```
GNU nano 8.7.1 file_syscall.py
import os
fd = os.open("sample.txt", os.O_CREAT | os.O_WRONLY, 0o644)
data = b"Hello from Linux system calls!"
os.write(fd, data)
os.close(fd)
print("Data written successfully")
```

[Read 6 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	^U Undo	^M-A Set Mark
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	^M-E Redo	^M-C Copy

Screenshot 2

Experiment 5 – Reading a File Using System Calls

Description: -

This experiment demonstrates reading file contents using `os.open()`, `os.read()`, and `os.close()`. The data is decoded and displayed after being read from `sample.txt`.

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano read_file.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 read_file.py
File contents:
Hello from Linux system calls!
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ |
```

Screenshot 1



```
GNU nano 8.7.1 read_file.py
import os
fd = os.open("sample.txt", os.O_RDONLY)
data = os.read(fd, 100)
os.close(fd)
print("File contents:")
print(data.decode())
```

[Read 6 lines]

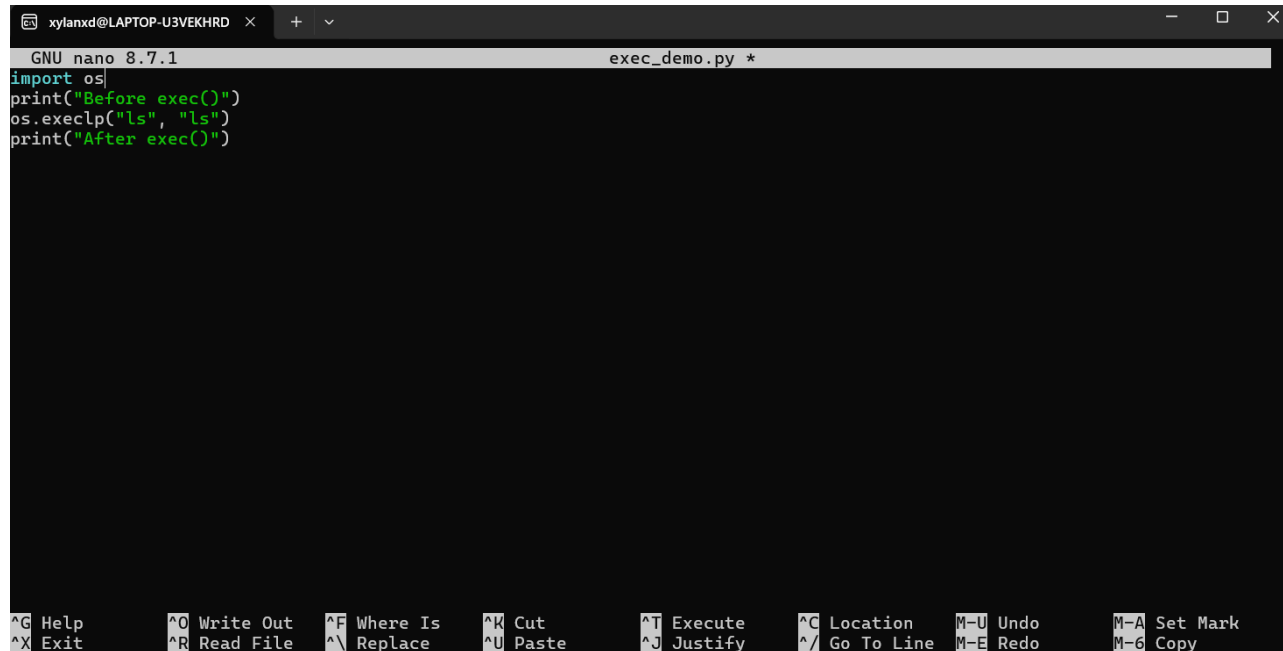
^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo	M-6 Copy

Screenshot 2

Experiment 6 - exec() System Call

Description: -

This experiment demonstrates how `exec()` replaces the current program with another program. The example replaces the Python process with the `ls` command, so the statement after `exec()` is not executed.



```
GNU nano 8.7.1                                exec_demo.py *
import os
print("Before exec()")
os.execlp("ls", "ls")
print("After exec()")
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-G Copy

Screenshot 1

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano exec_demo.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 exec_demo.py
Before exec()
exec_demo.py  file_syscall.py  fork_demo.py  process_id.py  read_file.py  sample.txt  wait_demo.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$
```

Screenshot 2

Experiment 7 - Combined Process Creation

Description: -

This combined experiment demonstrates process creation and synchronization using `fork()`, `getpid()`, `getppid()`, and `wait()`. It shows the parent and child processes and waits for the child to complete.

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ nano process_creation.py
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ python3 process_creation.py
Program started
Original PID: 1182

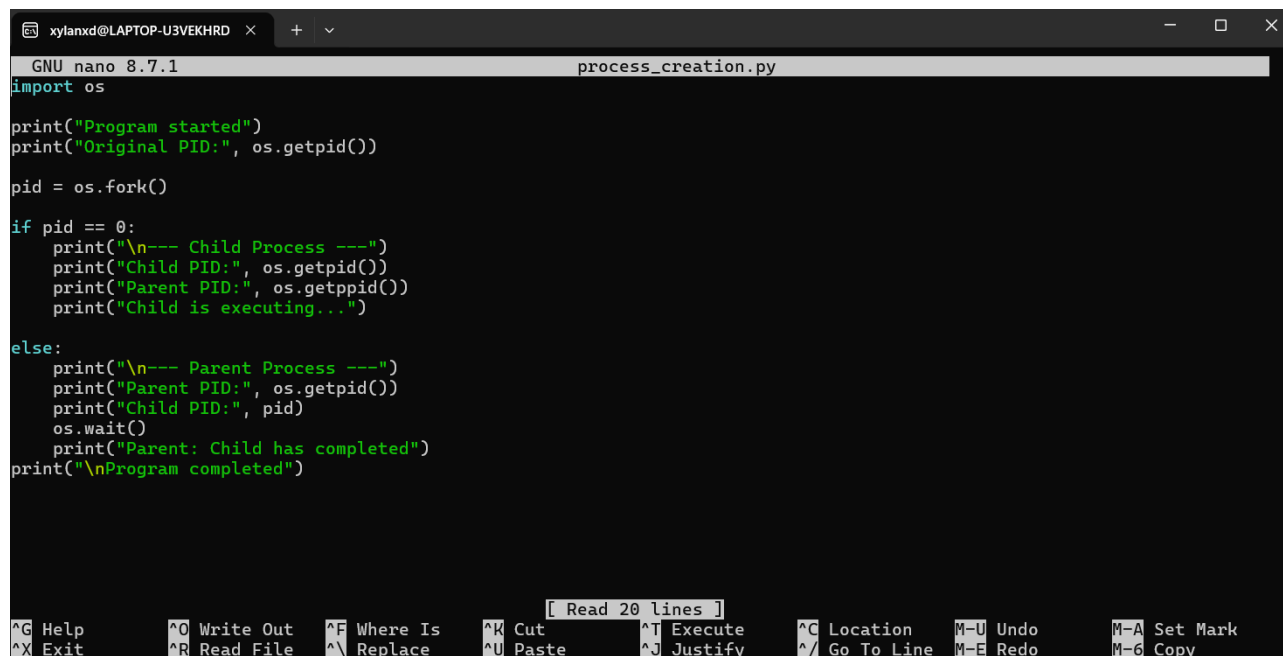
--- Parent Process ---
Parent PID: 1182
Child PID: 1183

--- Child Process ---
Child PID: 1183
Parent PID: 1182
Child is executing...

Program completed
Parent: Child has completed

Program completed
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ |
```

Screenshot 1



```
GNU nano 8.7.1 process_creation.py
import os

print("Program started")
print("Original PID:", os.getpid())

pid = os.fork()

if pid == 0:
    print("\n--- Child Process ---")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
    print("Child is executing...")
else:
    print("\n--- Parent Process ---")
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)
    os.wait()
    print("Parent: Child has completed")
print("\nProgram completed")

[ Read 20 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Screenshot 2

Project Branch / Folder Structure

The following screenshot shows the organized folder structure containing the seven experiments.

```
xylanxd@LAPTOP-U3VEKHRD:~/os_system_calls$ find . -maxdepth 1 -type f -printf '%f\n'
process_id.py
wait_demo.py
process_creation.py
sample.txt
read_file.py
file_syscall.py
fork_demo.py
exec_demo.py
```

Branch / folder overview